

Projet 6 : Conception et Mise en Œuvre d'un Entrepôt de Données Météorologiques

Formation : Data Engineering–Force-N

Mentor : MEDJEU FOPA Junior

Ibrahima FALL – Groupe 6



Table des matières

I. Contexte et Objectifs du Projet :	3
II. Collecte de données historique et création de la base de données source :	3
1. Outils utilisés :	3
2. Restauration et Constitution de la Base de Données Source :	4
3. Optimisations Techniques Appliquées :	6
4. Collecte Automatisée des Données Historiques :	6
5. Justification Technique :	9
III. Modélisation d'un entrepôt pour le suivi des données météo :	10
1. Choix du Modèle :	10
2. Identification des tables de dimensions:	11
3. Identification de la table de faits :	11
IV. Création de l'Entrepôt – Scripts SQL :	11
V. Chargement Initial (Chargement 1) :	12
2. Insertion des Faits Consolidés :	13
VI. Chargement Actualisé (Chargement 2) :	14
1. Script script2.sql :	14
2. Avantages de cette approche :	14
VII. Pipeline d'Automatisation (Chargement 3) :	15
1. Objectif du Chargement 3 :	15
2. Architecture Technique :	15
3. Composition du Pipeline :	16
4. Orchestration avec Airflow :	18
5. Vérification de la Collecte et Insertion des Données :	18
6. Avantages :	21
VIII. Validation Fonctionnelle de l'Entrepôt :	21
1. Objectif de la validation :	21
2. Méthodologie :	21
3. Résultats :	22
IX. Tableau de Bord (Power BI) :	26
1. Connexion PostgreSQL :	26
2. Création des visualisations :	30
3. Utilité :	37
X. Conclusion :	38

I. Contexte et Objectifs du Projet :

ClimaTech est une organisation spécialisée dans l'analyse des données météorologiques pour accompagner les agriculteurs, les acteurs du transport et les décideurs publics.

Face aux enjeux climatiques croissants, ClimaTech souhaite :

- Centraliser les données météo collectées par ses stations (température, précipitations, humidité, vent, etc.)
- Mettre en place un entrepôt de données robuste et évolutif
- Permettre des analyses fines et automatisées via des tableaux de bord décisionnels
- Améliorer la compréhension des tendances météo pour anticiper les risques



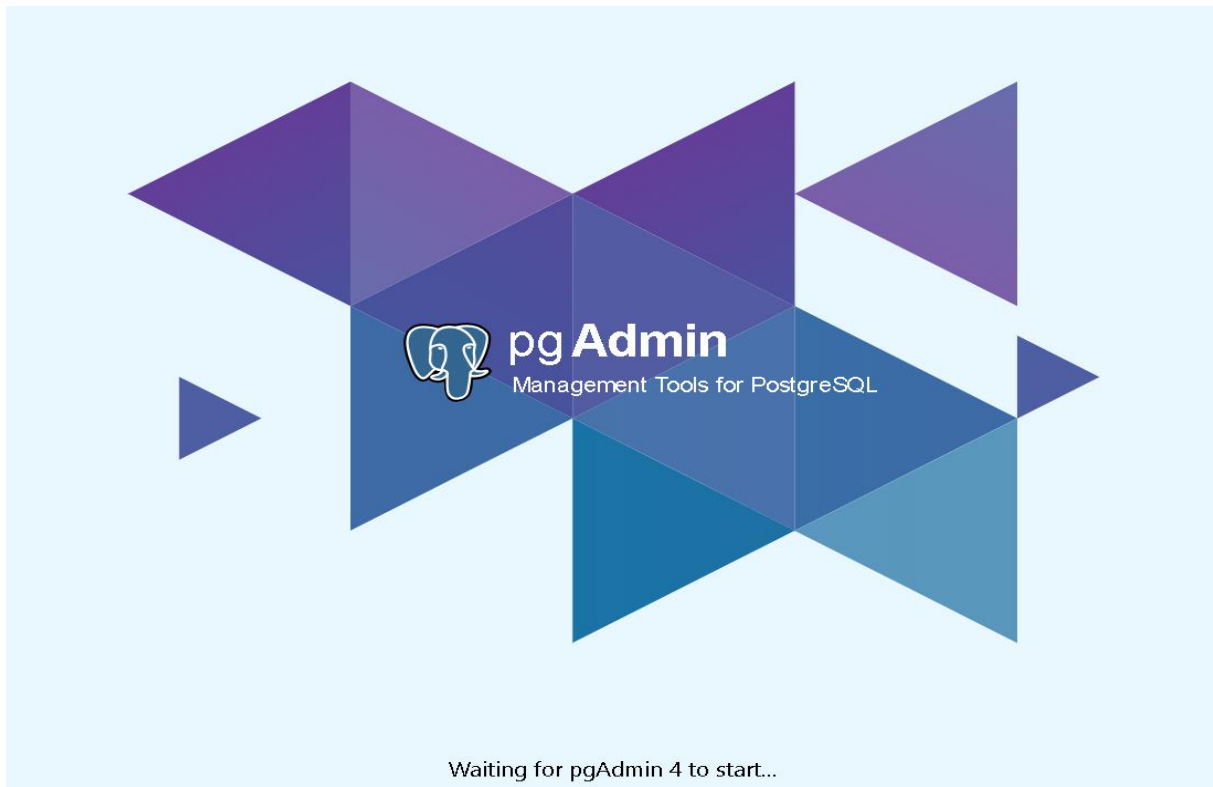
II. Collecte de données historique et création de la base de données source :

1. Outils utilisés :

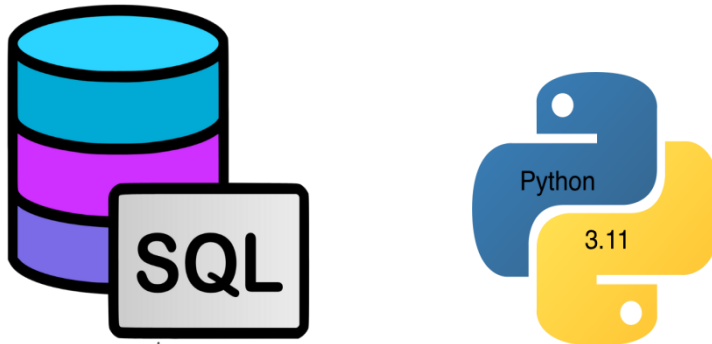
- **Système de gestion de base de données : PostgreSQL 17**



- **Client graphique : pgAdmin 4**



- **Langages : SQL et Python 3.11**



- **Bibliothèques Python : requests, psycopg2, pandas, logging**

2. Restauration et Constitution de la Base de Données Source :

Dans le contexte initial, un **dump** (meteo_uemoa.dump) était censé être fourni par ClimaTech. Cependant, ce fichier étant **indisponible**, une stratégie **alternative** a été mise en œuvre.

La base de données a été recrée **manuellement**, la connexion établie, puis la table brute **meteo_uemoa** a été créée pour accueillir les données météorologiques collectées :

The screenshot shows the pgAdmin 4 interface. In the top left, the 'Object Explorer' pane shows a tree structure: Servers (1) > PostgreSQL 17 > Databases (5) > climat_uemoa. The 'climat_uemoa' database is selected. The top right pane shows the SQL editor with the following SQL code:

```
1  -- Database: climat_uemoa
2
3  -- DROP DATABASE IF EXISTS climat_uemoa;
4
5  CREATE DATABASE climat_uemoa
6  WITH
7    OWNER = postgres
8    ENCODING = 'UTF8'
9    LC_COLLATE = 'fr-FR'
10   LC_CTYPE = 'fr-FR'
11   LOCALE_PROVIDER = 'libc'
12   TABLESPACE = pg_default
13   CONNECTION LIMIT = -1
14   IS_TEMPLATE = False;
```

The bottom pane shows the 'Query' editor with the following SQL code:

```
1  CREATE TABLE meteo_uemoa (
2    id SERIAL PRIMARY KEY, -- Identifiant unique auto-incrémenté pour chaque enregistrement
3    pays VARCHAR(50) NOT NULL, -- Nom du pays (par exemple : "Sénégal"), obligatoire
4    region VARCHAR(100) NOT NULL, -- Nom de la région ou de la station (ex : "Dakar"), obligatoire
5    latitude NUMERIC(8,5) NOT NULL, -- Latitude géographique en degrés, précision jusqu'à 5 décimales
6    longitude NUMERIC(8,5) NOT NULL, -- Longitude géographique en degrés, précision jusqu'à 5 décimales
7    altitude INTEGER NOT NULL, -- Altitude de la station en mètres
8    observation_time TIMESTAMP NOT NULL, -- Date et heure de l'observation météo (format standard ISO), obligatoire
9    temperature_2m REAL, -- Température à 2 mètres du sol en °C (valeur décimale)
10   relativehumidity_2m REAL, -- Humidité relative à 2 mètres en pourcentage (%)
11   dewpoint_2m REAL, -- Température du point de rosée en °C
12   pressure_msl REAL, -- Pression atmosphérique au niveau moyen de la mer en hPa
13   windspeed_10m REAL, -- Vitesse du vent à 10 mètres en km/h ou m/s
14   winddirection_10m REAL, -- Direction du vent à 10 mètres en degrés
15   precipitation REAL, -- Précipitations en mm
16   weathercode INTEGER, -- Code météo symbolique (ex : 0 pour ciel dégagé, 61 pour pluie légère, etc.)
17   cloudcover INTEGER, -- Couverture nuageuse en pourcentage (%)
18   visibility REAL, -- Visibilité en kilomètres (peut être nulle selon la source)
19   shortwave_radiation REAL, -- Rayonnement solaire de courte longueur d'onde (W/m²)
20   snowfall REAL, -- Chutes de neige en mm
21   collecte_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- Date et heure de l'insertion dans la base (ajoutée automatiquement)
22 );
23
```

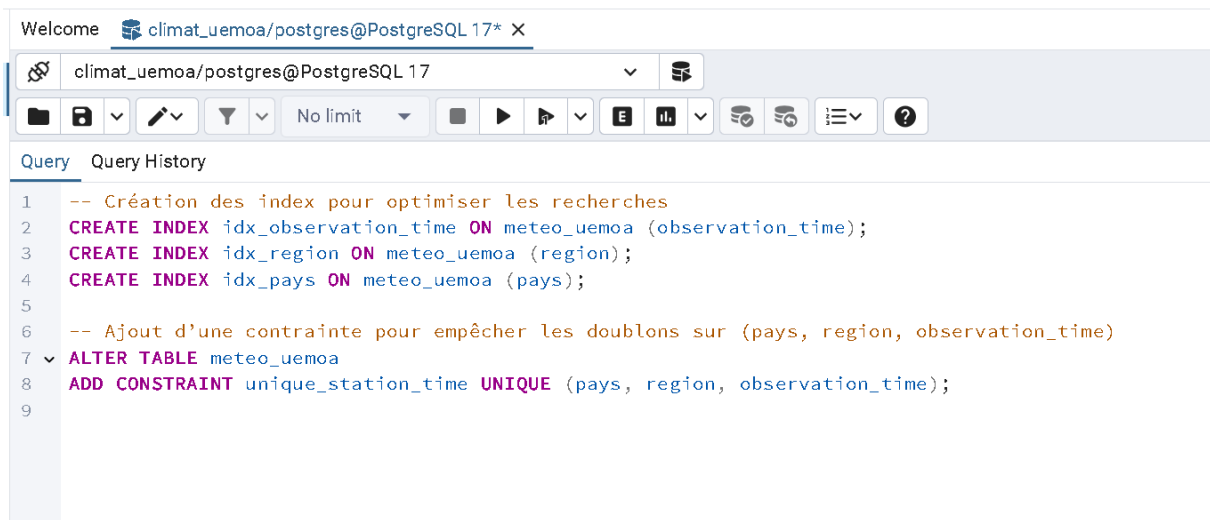
- Cette structure permet de stocker les **données météo brutes** collectées par **région**, avec des mesures **standards reconnues** (température, humidité, vent, etc.).

- Les champs **NOT NULL** garantissent que certaines informations **essentiels** (localisation, heure) sont toujours **présentes**.

- Le champ **collecte_timestamp** enregistre automatiquement la date et l'heure d'**insertion** de l'enregistrement, utile pour **tracer** l'origine des **données**.

- L'utilisation de types **précis** comme **NUMERIC(8,5)** pour la **latitude/longitude** permet une **précision** suffisante pour l'analyse **géographique**.

3. Optimisations Techniques Appliquées :



```
1  -- Création des index pour optimiser les recherches
2  CREATE INDEX idx_observation_time ON meteo_uemoa (observation_time);
3  CREATE INDEX idx_region ON meteo_uemoa (region);
4  CREATE INDEX idx_pays ON meteo_uemoa (pays);
5
6  -- Ajout d'une contrainte pour empêcher les doublons sur (pays, region, observation_time)
7  ALTER TABLE meteo_uemoa
8  ADD CONSTRAINT unique_station_time UNIQUE (pays, region, observation_time);
9
```

- **Index temporel :**

```
CREATE INDEX idx_observation_time ON meteo_uemoa (observation_time);
```

- **Index géographiques :**

```
CREATE INDEX idx_pays ON meteo_uemoa (pays);
CREATE INDEX idx_region ON meteo_uemoa (region);
```

- **Contrainte d'unicité :** Garantit l'absence de doublons pour un relevé donné par station et par heure :

```
ALTER TABLE meteo_uemoa
ADD CONSTRAINT unique_station_time UNIQUE (pays, region, observation_time);
```

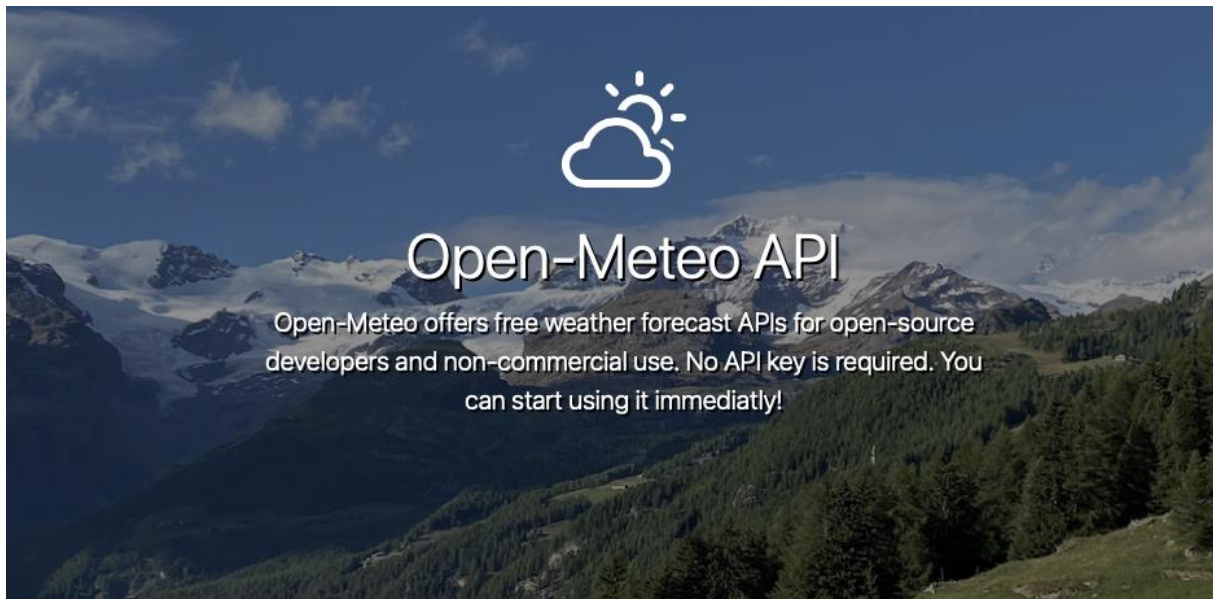
4. Collecte Automatisée des Données Historiques :

a) Source des données :

L'API **Open-Meteo Archives** a été utilisée pour collecter les données météorologiques horaires historiques couvrant l'ensemble des **pays** de l'**UEMOA**. Cette API fournit des données **libres** et **accessibles**, basées sur des **modèles** climatiques de **référence**, permettant d'obtenir plusieurs **variables météorologiques** telles que la température, l'humidité, la vitesse et la direction du vent, les précipitations, la couverture nuageuse, la pression atmosphérique, etc.

Dans le cadre de ce projet, les données ont été récupérées pour la période comprise entre **janvier 2023 et juin 2025**, avec une résolution horaire, ce qui offre un niveau de détail suffisant pour effectuer des **analyses** fines, telles que l'étude des tendances climatiques ou l'impact des conditions météorologiques sur les activités socio-économiques.

La collecte a été réalisée **automatiquement** via un **script Python** interrogeant l'**API** pour chaque **région** géographique **définie** (latitude, longitude, altitude), avec un **stockage simultané** en **base** de données et dans un **fichier CSV** consolidé.



b) Script Python développé :

Dans le cadre de la récupération des données météorologiques pour l'UEMOA, un **script Python robuste** et **automatisé** a été développé. Celui-ci intègre l'ensemble des fonctionnalités **nécessaires** pour assurer une **collecte fiable**, un stockage **sécurisé** et un **archivage** hors base de données. Les principales **caractéristiques** sont les suivantes :

Requête API Open-Meteo : Le script interroge automatiquement l'API d'archives météorologiques pour chaque région des **pays** membres de l'**UEMOA** (Sénégal, Côte d'Ivoire, Guinée-Bissau, Mali, Niger, Togo, etc.), en utilisant leurs **coordonnées géographiques** (latitude, longitude, altitude) :

```
    {"nom": "Plateaux", "latitude": 7.5333, "longitude": 1.8500, "altitude": 350, "pays": "Togo"},  
    {"nom": "Savanes", "latitude": 10.3500, "longitude": 0.8333, "altitude": 200, "pays": "Togo"}  
]  
}  
  
# ----- Fonctions Utilitaires -----  
  
def fetch_historical_weather(lat, lon, start_date, end_date):  
    """  
    Interroge l'API Open-Meteo Archive pour récupérer les données météo historiques.  
    """  
    url = "https://archive-api.open-meteo.com/v1/archive"  
    params = {  
        "latitude": lat,  
        "longitude": lon,  
        "start_date": start_date.strftime("%Y-%m-%d"),  
        "end_date": end_date.strftime("%Y-%m-%d"),  
        "hourly": ", ".join(HOURLY_VARIABLES),  
        "timezone": "Africa/Abidjan"  
    }  
    try:
```

Gestion avancée des erreurs et des interruptions : Le script intègre des mécanismes de gestion des exceptions, avec des journaux détaillés (**logging**) permettant d'identifier les anomalies, les erreurs de connexion, ou les échecs d'insertion.

Export simultané au format CSV : Parallèlement à l'insertion en base, les données sont archivées dans un fichier **archives_meteo_uemoa.csv**. Ce fichier permet de disposer d'une sauvegarde **externe**, indépendante de la base, facilitant les **traitements** hors-ligne et les vérifications manuelles.

archives meteo uemoa	29/06/2025 16:34	Fichier CSV Microsoft...	41242 Ko
----------------------	------------------	--------------------------	----------

Respect des limitations API : Une pause automatique d'**1 seconde** est appliquée **entre** chaque **requête** afin de ne **pas saturer** les serveurs de l'**API** et de se conformer aux bonnes **pratiques** d'accès aux données **ouvertes** :

```
logging.info(f"Collecte terminée pour {region['nom']}")
time.sleep(1)
```

Conception modulaire et évolutive : Le script repose sur des **fonctions** réutilisables (fetch_historical_weather, insert_record, export_to_csv) et peut **facilement** être étendu à d'autres pays ou d'autres variables météorologiques.

Ce dispositif constitue la **pierre angulaire** de l'alimentation de la base de données **brute**, préalable **indispensable** à la construction de l'entrepôt de données **météo**.

c) Exemple de sortie après exécution du code python :

```
2025-06-29 14:58:17 [INFO] Collecte terminée pour Niamey
2025-06-29 14:58:18 [INFO] Collecte Togo - Centrale en cours...
2025-06-29 14:58:27 [INFO] Collecte terminée pour Centrale
2025-06-29 14:58:28 [INFO] Collecte Togo - Kara en cours...
2025-06-29 14:58:37 [INFO] Collecte terminée pour Kara
2025-06-29 14:58:38 [INFO] Collecte Togo - Maritime en cours...
2025-06-29 14:58:45 [INFO] Collecte terminée pour Maritime
2025-06-29 14:58:46 [INFO] Collecte Togo - Plateaux en cours...
2025-06-29 14:58:55 [INFO] Collecte terminée pour Plateaux
2025-06-29 14:58:56 [INFO] Collecte Togo - Savanes en cours...
2025-06-29 14:59:05 [INFO] Collecte terminée pour Savanes
2025-06-29 14:59:06 [INFO] Connexion à la base fermée
2025-06-29 14:59:12 [INFO] Export CSV terminé : 766296 lignes dans 'archives_meteo_uemoa.csv'

[Done] exited with code=0 in 807.578 seconds
```

5. Justification Technique :

La **stratégie** mise en œuvre pour la **constitution** de la base brute et la **collecte** automatisée répond à des **exigences précises** de **qualité**, de **fiabilité** et de **pérennité** des données, **essentiels** dans le cadre d'un projet d'entrepôt de données météorologiques.

Les choix techniques adoptés garantissent :

Une base consolidée, normalisée et bien structurée :

La table **meteo_uemoa** a été conçue pour accueillir l'**ensemble** des **données météorologiques** collectées, en respectant les bonnes pratiques de modélisation relationnelle. Les **types** de données sont **adaptés** (géographiques, temporels, numériques), les **contraintes** d'intégrité (NOT NULL, clé primaire, unicité) assurent la **cohérence** des enregistrements, et les **index** améliorent les **performances** lors des **recherches**.

Des données historiques homogènes et cohérentes sur l'ensemble de l'UEMOA :

Le processus **automatisé** garantit que les données sont collectées de manière uniforme pour toutes les **régions ciblées**, avec les mêmes **variables** météorologiques et la même granularité **horaire**. Cela permet d'éliminer les biais potentiels et d'assurer la **comparabilité** des mesures entre les différents territoires.

L'élimination des doublons et la fiabilité des insertions :

Grâce à la **contrainte** d'unicité SQL (pays, région, observation_time), les doublons sont

automatiquement rejetés lors de l'insertion, garantissant ainsi que chaque relevé horaire par région est **unique**. Cela évite les **erreurs** de consolidation et simplifie les traitements en aval.

Une traçabilité complète et une sauvegarde externe des données :

Le champ **collecte_timestamp** enregistre automatiquement la **date** et l'**heure d'insertion**, ce qui permet de tracer l'**origine** des **données** et de distinguer les **éventuelles** mises à jour ultérieures. L'export parallèle au format **CSV** fournit une **copie** indépendante, utile pour les **sauvegardes**, les **échanges** de données ou les **vérifications** hors-ligne.

Une solution robuste et évolutive :

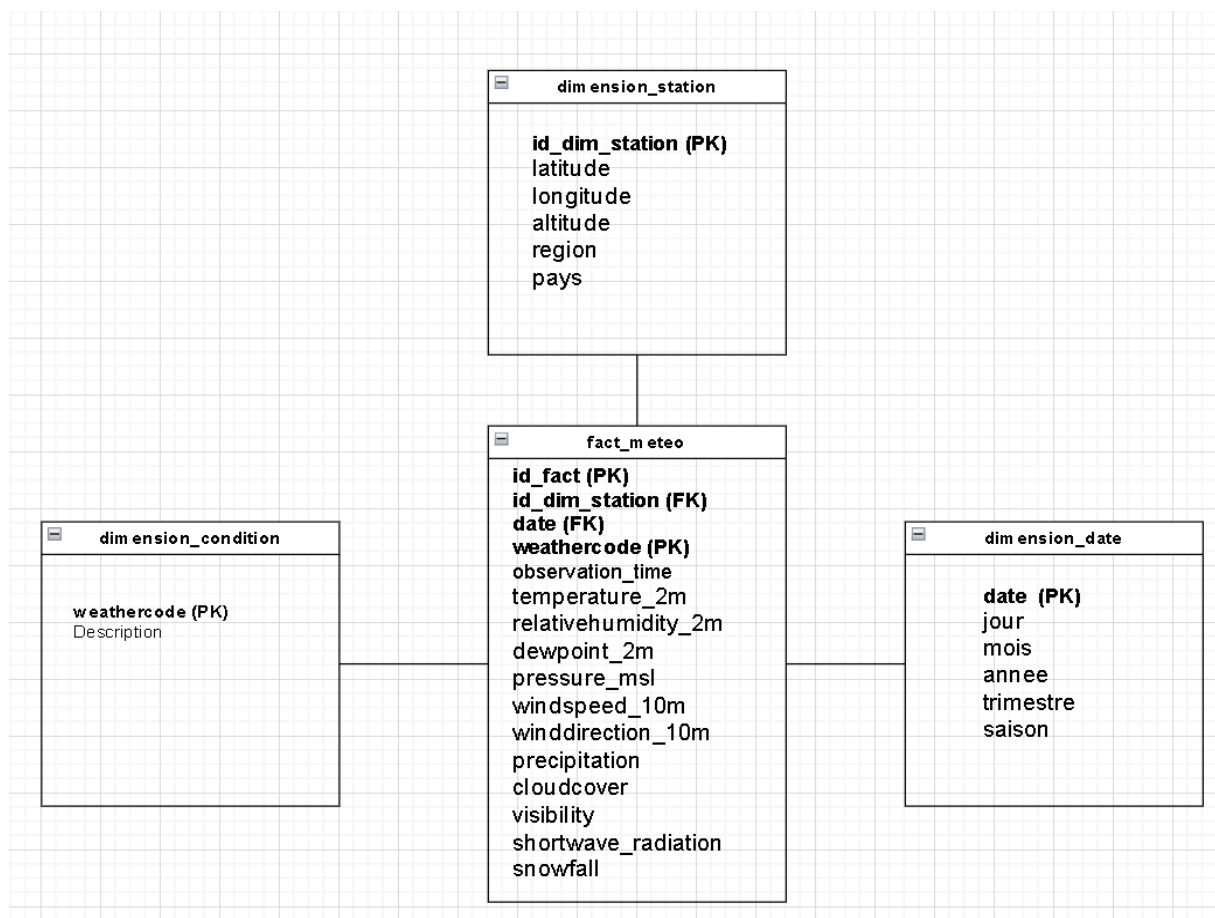
Le **script** Python est conçu de manière modulaire, avec des **fonctions** réutilisables et des **paramètres** facilement ajustables. Cela facilite l'extension **future** à d'autres **régions**, **pays** ou **variables**, sans remise en cause de l'architecture **existante**.

Cette méthode garantit une **source** de données **fiable**, **cohérente** et **exploitable**, constituant ainsi une base **solide** pour la **modélisation** en étoile et l'alimentation **progressive** de l'**entrepôt** de **données**.

III. Modélisation d'un entrepôt pour le suivi des données météo :

1. Choix du Modèle :

Le projet repose sur un **schéma en étoile**, architecture classique et efficace des entrepôts de données. Ce **modèle** est particulièrement **adapté** aux besoins **analytiques** :



. **Facilite les jointures** grâce à la centralisation des mesures dans une table de faits et la séparation des dimensions.

. **Optimise les requêtes d'agrégation** (moyennes, totaux, tendances) sur les données météorologiques.

. **Simplifie la lecture et la maintenance du modèle** par sa structure claire et normalisée.

2. Identification des tables de dimensions:

Trois dimensions ont été modélisées :

dimension_station :

Stocke les métadonnées **géographiques** et **administratives** des stations :

- **id_dim_station** : identifiant unique de la station
- **pays, region** : informations administratives
- **latitude, longitude, altitude** : coordonnées précises

dimension_date :

Permet l'analyse temporelle sous plusieurs granularités :

- **date** : clé primaire (jour calendaire)
- **jour, mois, annee, trimestre, saison** : détails dérivés

dimension_condition :

Associe un **weathercode** à sa description textuelle pour mieux **interpréter** les conditions **météo** (pluie, brouillard, etc.).

3. Identification de la table de faits :

Cette table **centralise** l'ensemble des **mesures** météorologiques brutes :

- Clé étrangère vers **chaque dimension** (station, date, condition)
- Stockage des **variables quantitatives** observées : température, humidité, vent, précipitations, etc.
- Champ **observation_time** pour les analyses **horaires précises**

Cette modélisation garantit **flexibilité, rapidité** des analyses et **évolutivité** du système.

IV. Création de l'Entrepôt – Scripts SQL :

La constitution physique de l'entrepôt a été réalisée via le script SQL suivant :

```
3_Script_SQL_CreationEntrepotDonnees.sql X
C:\Users\FT\Downloads\Projet6 Groupe6 DataEngenering > Etape3_Création de l'entrepôt > 3_Script_SQL_CreationEntrepotDonnees.sql
1  -- Dimension Station (stations météo de l'UEMOA)
2  CREATE TABLE dimension_station (
3      id_dim_station SERIAL PRIMARY KEY,
4      pays VARCHAR(50),
5      region VARCHAR(100),
6      latitude NUMERIC(8,5),
7      longitude NUMERIC(8,5),
8      altitude INTEGER,
9      UNIQUE (pays, region) -- Garantie d'une station unique par région
10 );
11
12 -- Dimension Date (analyse temporelle)
13 CREATE TABLE dimension_date (
14     date DATE PRIMARY KEY,
15     jour INTEGER,
16     mois INTEGER,
17     annee INTEGER,
18     trimestre INTEGER,
19     saison VARCHAR(20)
20 );
21
22 -- Dimension Condition (codes météo standardisés)
23 CREATE TABLE dimension_condition (
24     weathercode INTEGER PRIMARY KEY,
25     description VARCHAR(100)
26 );
27
3_Script_SQL_CreationEntrepotDonnees.sql X
C:\Users\FT\Downloads\Projet6 Groupe6 DataEngenering > Etape3_Création de l'entrepôt > 3_Script_SQL_CreationEntrepotDonnees.sql
28 -- Table de Faits Météo (mesures consolidées)
29 CREATE TABLE fact_meteo (
30     id_fact SERIAL PRIMARY KEY,
31     id_dim_station INTEGER REFERENCES dimension_station(id_dim_station),
32     date DATE REFERENCES dimension_date(date),
33     weathercode INTEGER REFERENCES dimension_condition(weathercode),
34     observation_time TIMESTAMP,
35     temperature_2m REAL,
36     relativehumidity_2m REAL,
37     dewpoint_2m REAL,
38     pressure_msl REAL,
39     windspeed_10m REAL,
40     winddirection_10m REAL,
41     precipitation REAL,
42     cloudcover INTEGER,
43     visibility REAL,
44     shortwave_radiation REAL,
45     snowfall REAL
46 );
47
```

Optimisations Techniques Appliquées :

- **Contraintes d'intégrité** via **clés** primaires et étrangères pour garantir la cohérence des données.
- **Index supplémentaires** sur les champs critiques (date, id_dim_station) permettant d'accélérer les **jointures** et **filtres** lors des requêtes analytiques.
- **Unicité des stations** assurée par la contrainte **UNIQUE(pays, region)** dans la dimension station, évitant la redondance des enregistrements géographiques.

V. Chargement Initial (Chargement 1) :

1. Script script1.sql :

Le **premier chargement** de l'entrepôt consiste à :

- **Alimenter** les **dimensions** à partir de la table source meteo_uemoa
- **Éviter** les **doublons** grâce à la clause **ON CONFLICT DO NOTHING**
- Effectuer les **jointures** nécessaires pour **insérer** les données **consolidées** dans la table de faits fact_meteo

Extrait du script :

```
script1.sql X
C:\Users\FT > Downloads > Projet6 Groupe6 DataEngineering > Etape4_Chargement 1 > script1.sql
1  -- Chargement dimension station (extrait unique des stations)
2  INSERT INTO dimension_station (pays, region, latitude, longitude, altitude)
3  SELECT DISTINCT pays, region, latitude, longitude, altitude
4  FROM meteo_uemoa
5  ON CONFLICT (pays, region) DO NOTHING;
6
7  -- Chargement dimension date (extrait unique des dates, calcul attributs)
8  INSERT INTO dimension_date (date, jour, mois, annee, trimestre, saison)
9  SELECT DISTINCT
10     DATE(observation_time),
11     EXTRACT(DAY FROM observation_time)::INT,
12     EXTRACT(MONTH FROM observation_time)::INT,
13     EXTRACT(YEAR FROM observation_time)::INT,
14     EXTRACT(QUARTER FROM observation_time)::INT,
15     CASE
16         WHEN EXTRACT(MONTH FROM observation_time) IN (12, 1, 2) THEN 'Hiver'
17         WHEN EXTRACT(MONTH FROM observation_time) IN (3, 4, 5) THEN 'Printemps'
18         WHEN EXTRACT(MONTH FROM observation_time) IN (6, 7, 8) THEN 'Été'
19         ELSE 'Automne'
20     END AS saison
21 FROM meteo_uemoa
22 ON CONFLICT (date) DO NOTHING;
23
24 -- Chargement dimension condition météo (à faire une fois) --
25
26 INSERT INTO dimension_condition (weathercode, description) VALUES
27 (0, 'Ciel dégagé'),
28 (1, 'Principalement ensoleillé'),
29 (2, 'Partiellement nuageux'),
30 (3, 'Couvert'),
```

La dimension **dimension_condition** étant basée sur un référentiel **standard**, son chargement s'effectue une **seule fois** en insérant les couples (**weathercode, description**) fournis par Open-Meteo.

2. Insertion des Faits Consolidés :

Les données météorologiques brutes sont **insérées** dans la table **fact_meteo** après résolution des **clés étrangères** :

```
script1.sql X
C:\Users\FT > Downloads > Projet6 Groupe6 DataEngineering > Etape4_Chargement 1 > script1.sql
51 (86, 'Averses de neige fortes'),
52 (95, 'Orage (léger ou modéré)'),
53 (96, 'Orage avec grêle légère'),
54 (99, 'Orage avec grêle forte');
55
56 -- Chargement de la table de faits
57 INSERT INTO fact_meteo (
58     id_dim_station, date, weathercode, observation_time,
59     temperature_2m, relativehumidity_2m, dewpoint_2m, pressure_msl,
60     windspeed_10m, winddirection_10m, precipitation, cloudcover,
61     visibility, shortwave_radiation, snowfall
62 )
63 SELECT
64     s.id_dim_station,
65     DATE(m.observation_time),
66     m.weathercode,
67     m.observation_time,
68     m.temperature_2m, m.relativehumidity_2m, m.dewpoint_2m, m.pressure_msl,
69     m.windspeed_10m, m.winddirection_10m, m.precipitation, m.cloudcover,
70     m.visibility, m.shortwave_radiation, m.snowfall
71 FROM meteo_uemoa m
72 JOIN dimension_station s ON m.pays = s.pays AND m.region = s.region
73 JOIN dimension_date d ON DATE(m.observation_time) = d.date;
74
75 -- Après pour charger des données actualisées tout en évitant les doublons, on utilise ON CONFLICT DO NOTHING --
76
77 -- Ajoute une contrainte d'unicité sur la table de faits (Faire une fois) --
78 ALTER TABLE fact_meteo
79 ADD CONSTRAINT unique_fait_station_time UNIQUE (id_dim_station, observation_time);
80
```

Remarque : Une **contrainte d'unicité** peut être ajoutée sur (**id_dim_station, observation_time**) pour renforcer la **qualité** des données :

- Ce **premier** chargement permet d'exploiter **immédiatement** l'entrepôt via des **requêtes** d'agrégation ou d'analyse.

VI. Chargement Actualisé (Chargement 2) :

1. Script script2.sql :

Le **second** chargement vise à intégrer les **nouvelles** données **collectées** après le chargement **initial**, sans réintroduire de **doublons** :

- **Mise à jour** incrémentale des **dimensions** : insertion **uniquement** des **nouvelles** stations ou dates

- **Alimentation** de la table de **faits** avec gestion des doublons via **ON CONFLICT**

Extrait du script :

```
script2.sql X
C: > Users > FT > Downloads > Projet6 Groupe6 DataEngineering > Etape5_Chargement 2 > script2.sql
1  -- Chargement dimension date
2  INSERT INTO dimension_date (date, jour, mois, annee, trimestre, saison)
3  SELECT DISTINCT
4      DATE(observation_time),
5      EXTRACT(DAY FROM observation_time)::INT,
6      EXTRACT(MONTH FROM observation_time)::INT,
7      EXTRACT(YEAR FROM observation_time)::INT,
8      EXTRACT(QUARTER FROM observation_time)::INT,
9      CASE
10         WHEN EXTRACT(MONTH FROM observation_time) IN (12, 1, 2) THEN 'Hiver'
11         WHEN EXTRACT(MONTH FROM observation_time) IN (3, 4, 5) THEN 'Printemps'
12         WHEN EXTRACT(MONTH FROM observation_time) IN (6, 7, 8) THEN 'Été'
13         ELSE 'Automne'
14     END AS saison
15 FROM meteo_uemoa
16 ON CONFLICT (date) DO NOTHING;
17
18 -- Chargement des faits
```

```
script2.sql X
C: > Users > FT > Downloads > Projet6 Groupe6 DataEngineering > Etape5_Chargement 2 > script2.sql
18 -- Chargement des faits --
19 INSERT INTO fact_meteo (
20     id_dim_station, date, weathercode, observation_time,
21     temperature_2m, relativehumidity_2m, dewpoint_2m, pressure_msl,
22     windspeed_10m, winddirection_10m, precipitation, cloudcover,
23     visibility, shortwave_radiation, snowfall
24 )
25 SELECT
26     s.id_dim_station,
27     DATE(m.observation_time),
28     m.weathercode,
29     m.observation_time,
30     m.temperature_2m, m.relativehumidity_2m, m.dewpoint_2m, m.pressure_msl,
31     m.windspeed_10m, m.winddirection_10m, m.precipitation, m.cloudcover,
32     m.visibility, m.shortwave_radiation, m.snowfall
33 FROM meteo_uemoa m
34 JOIN dimension_station s ON m.pays = s.pays AND m.region = s.region
35 JOIN dimension_date d ON DATE(m.observation_time) = d.date
36 ON CONFLICT (id_dim_station, observation_time) DO NOTHING;
37
```

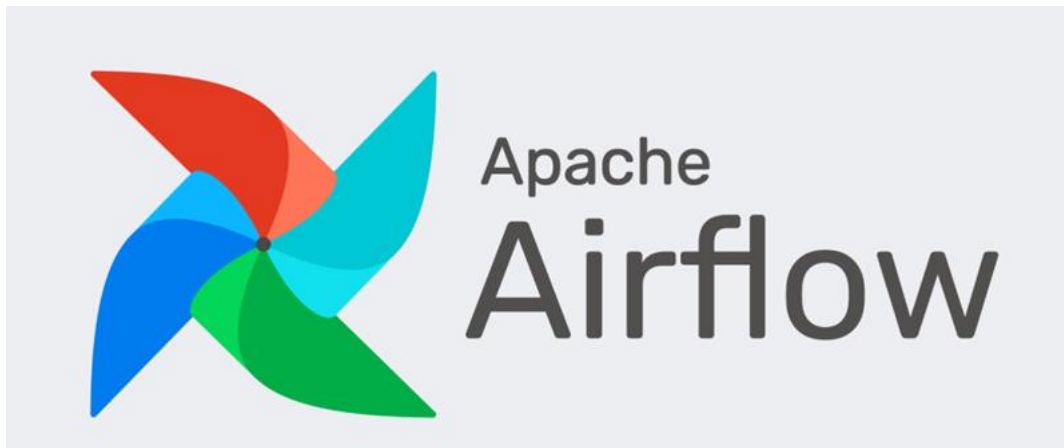
2. Avantages de cette approche :

- Permet une **alimentation régulière** sans risquer les doublons
- Compatible avec une **automatisation** via **Airflow** ou autre orchestrateur ETL

- Garantit l'**intégrité** et l'**homogénéité** des données dans l'entrepôt

VII. Pipeline d'Automatisation (Chargement 3) :

Un **pipeline d'automatisation complet** a été mis en place. Il est basé sur la plateforme **Apache Airflow**, permettant l'exécution sans intervention manuelle de la collecte et du traitement des données météorologiques pour l'ensemble des régions de l'espace UEMOA.



1. Objectif du Chargement 3 :

Ce pipeline permet d'automatiser :

- La collecte **horaire** des données **météo** en **temps réel** via l'API Open-Meteo
- L'insertion de ces données dans la base de données relationnelle **PostgreSQL** (table meteo_uemoa)
- L'exportation consolidée des données dans un fichier **CSV** à des fins d'**archivage**
- L'alimentation de l'entrepôt de données par **mise à jour** des **tables** dimension_date et fact_meteo

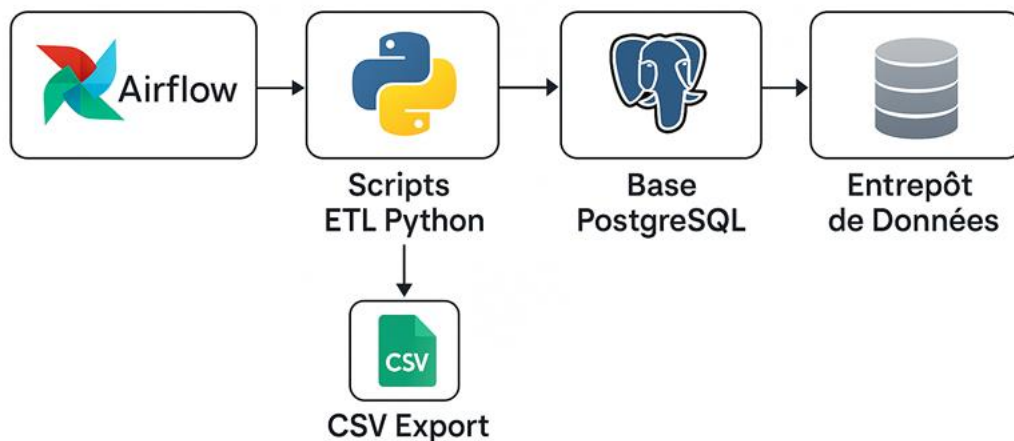
L'objectif est d'assurer une **disponibilité** continue et **fiable** des **données** météorologiques pour l'analyse **décisionnelle**.

2. Architecture Technique :

Le pipeline repose sur l'environnement suivant :

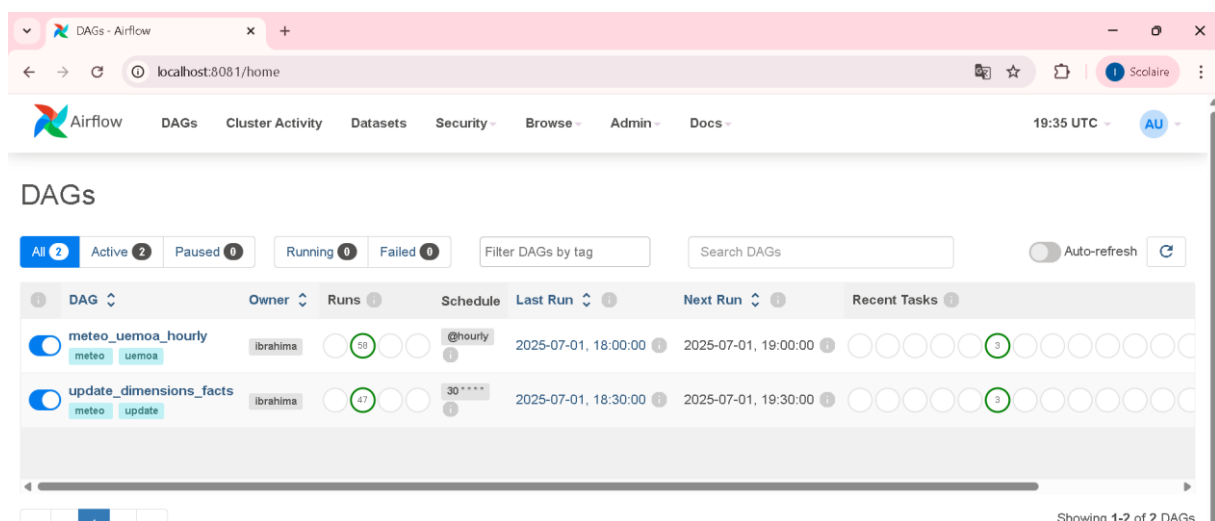
Composant	Description
Airflow	Orchestrateur de workflows (version 2.9.1)
Python	Langage principal des scripts ETL (3.11)
PostgreSQL	Base de données météo (climat_uemoa)
Docker	Conteneurisation des services Airflow/Postgres
API Open-Meteo	Fournisseur des données météo temps réel

Schéma simplifié :



3. Composition du Pipeline :

Le pipeline d'automatisation s'appuie sur **deux DAGs** principaux dans Airflow :



Pour assurer l'automatisation **complète** du projet, ces deux DAGs ont été développés sous **Apache Airflow**, comme illustré par les **extraits** de code. Le **premier** DAG, `meteo_uemoa_hourly`, exécute **automatiquement** la **collecte** des données météorologiques toutes les **heures**, en appelant les **fonctions** métiers `fetch_weather_current_hour` et `insert_into_postgres`. Le **second** DAG, `update_dimensions_facts`, se charge de **mettre à jour** les **dimensions** et la **table de faits** du **Data Warehouse** toutes les **30 minutes**. Les chemins d'accès aux **scripts** sont définis **dynamiquement**, et une **gestion des erreurs** a été intégrée pour garantir la **robustesse** du pipeline.

Extraits de code :


```

1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from datetime import datetime, timedelta
4 import sys, os, logging
5 import pandas as pd
6 import psycpg2
7
8 scripts_path = '/opt/airflow/scripts'
9 if scripts_path not in sys.path:
10     sys.path.append(scripts_path)
11
12 from etl_meteo import fetch_weather_current_hour, insert_into
13
14 default_args = {
15     'owner': 'ibrahima',
16     'depends_on_past': False,
17     'retries': 1,
18     'retry_delay': timedelta(minutes=5),
19 }
20
21 dag = DAG(
22     dag_id='meteo_uemoa_hourly',
23     default_args=default_args,
24     description='ETL météo UEMOA chaque heure - pipeline détaillé',
25     schedule_interval='@hourly',
26     start_date=datetime(2025, 1, 1),
27     catchup=False,
28     tags=['meteo', 'uemoa'],
29 )
30
31 TEMP_DATA_PATH = "/opt/airflow/exports/temp_data.parquet"
32 CSV_EXPORT_PATH = "/opt/airflow/exports/meteo_uemoa.csv"
33
34 def collect_data(**context):
35     all_records = fetch_weather_current_hour(

```

```

1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from datetime import datetime, timedelta
4 import sys
5
6 # Chemin vers les scripts
7 scripts_path = '/opt/airflow/scripts'
8 if scripts_path not in sys.path:
9     sys.path.append(scripts_path)
10
11 # Import des fonctions spécifiques
12 try:
13     from update_dimensions_facts import update_dimensions_facts
14 except ImportError as e:
15     print(f"[DAG LOAD ERROR] Impossible d'importer update_dimensions_facts")
16     raise
17
18 # Paramètres des tâches
19 default_args = {
20     'owner': 'ibrahima',
21     'depends_on_past': False,
22     'retries': 1,
23     'retry_delay': timedelta(minutes=10),
24 }
25
26 # Définition du DAG
27 with DAG(
28     dag_id='update_dimensions_facts',
29     default_args=default_args,
30     description='Mise à jour en 3 étapes : dimension_date, fact_meteo, fact_dimension',
31     schedule_interval='30 * * * *',
32     start_date=datetime(2025, 1, 1),
33     catchup=False,
34     tags=['meteo', 'update'],
35 ) as dag:
36     update_dimensions_facts = PythonOperator(
37         task_id='update_dimensions_facts',
38         python_callable=update_dimensions_facts,
39     )

```

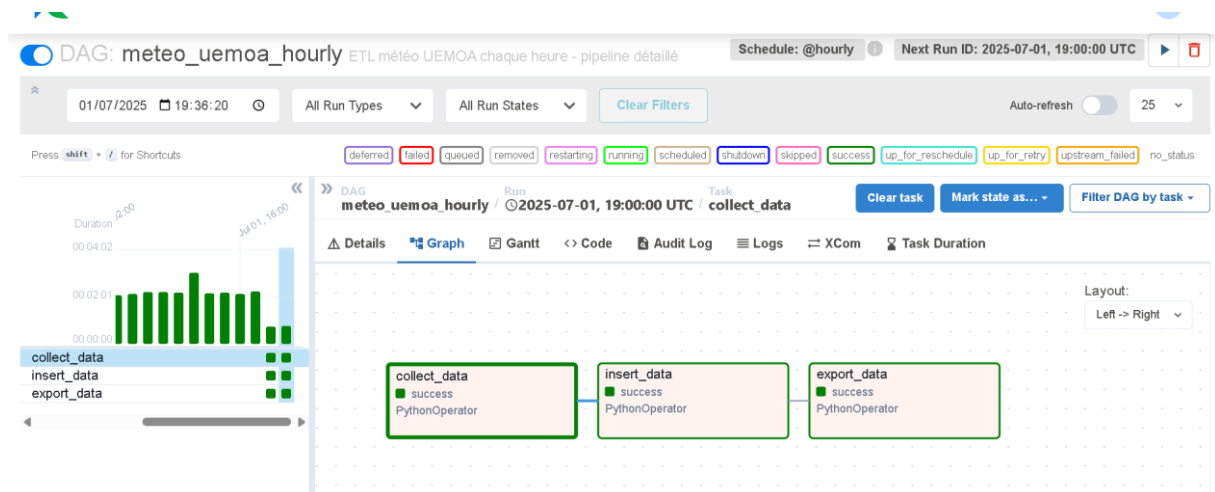
Ces automatisations permettent de **maintenir** la base de **données** et l'entrepôt **météo** synchronisés et **opérationnels** en temps réel.

a) DAG *meteo_uemoa_hourly* :

- Exécution **planifiée** chaque **heure**
- Tâches séquentielles :

- **Collecte des données météo** par appel à l'API Open-Meteo pour toutes les régions de l'UEMOA
- **Stockage temporaire** des résultats au format Parquet
- **Insertion** des enregistrements dans la base de données *meteo_uemoa*
- **Export consolidé** des données au format CSV (en évitant les doublons)

Visualisation dans Airflow :

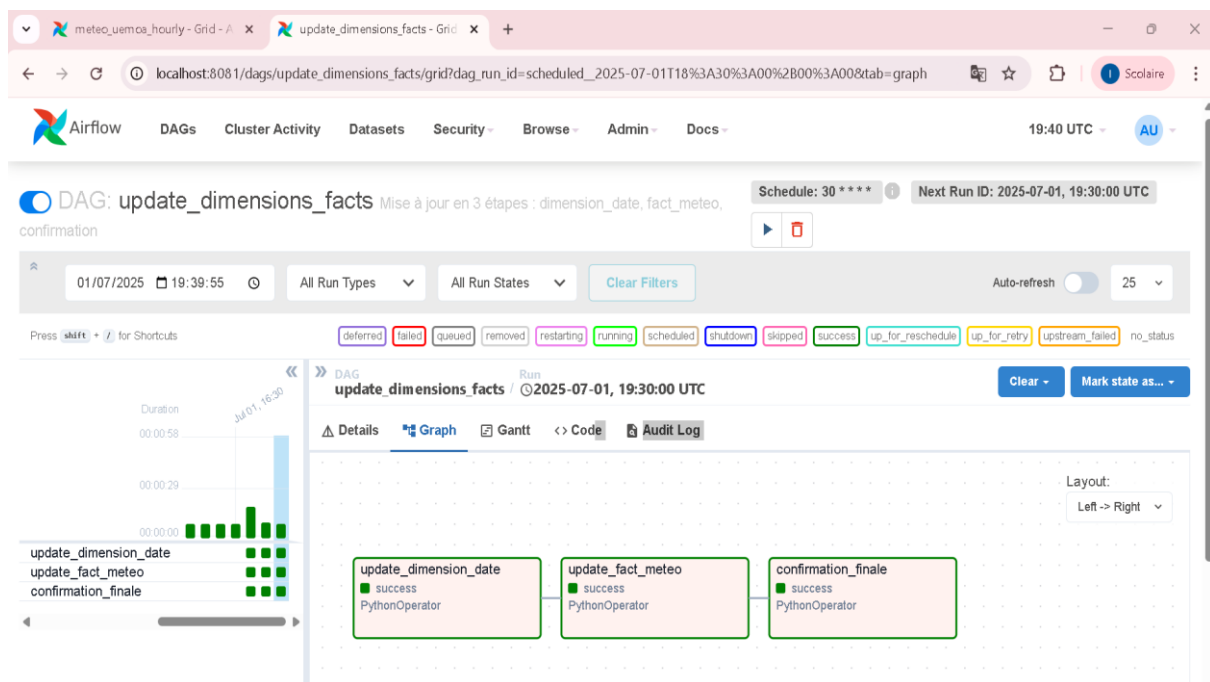


b) DAG *update_dimensions_facts* :

- Exécution planifiée à la **30^e minute** de chaque **heure**
- Tâches séquentielles :

- **Mise à jour de la dimension *dimension_date*** (extraction de la date, saison, trimestre...)
- **Alimentation de la table de faits *fact_meteo*** à partir des données consolidées
- **Tâche de confirmation finale**

Visualisation dans Airflow :



Cette séparation assure une meilleure **maîtrise** des **flux** et **réduit** les risques de **surcharge** de l'API.

4. Orchestration avec Airflow :

L'utilisation d'**Airflow** garantit :

- Une exécution **planifiée** et **automatisée**
- La **traçabilité** complète via l'interface web **Airflow** (logs, succès/échecs)
- La modularité et la maintenance **simplifiée** du pipeline
- La **persistance** des données météo et leur **intégration** dans l'entrepôt analytique

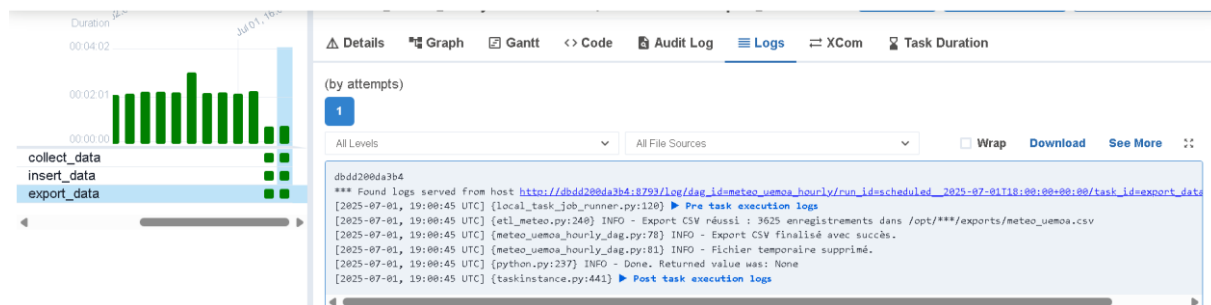
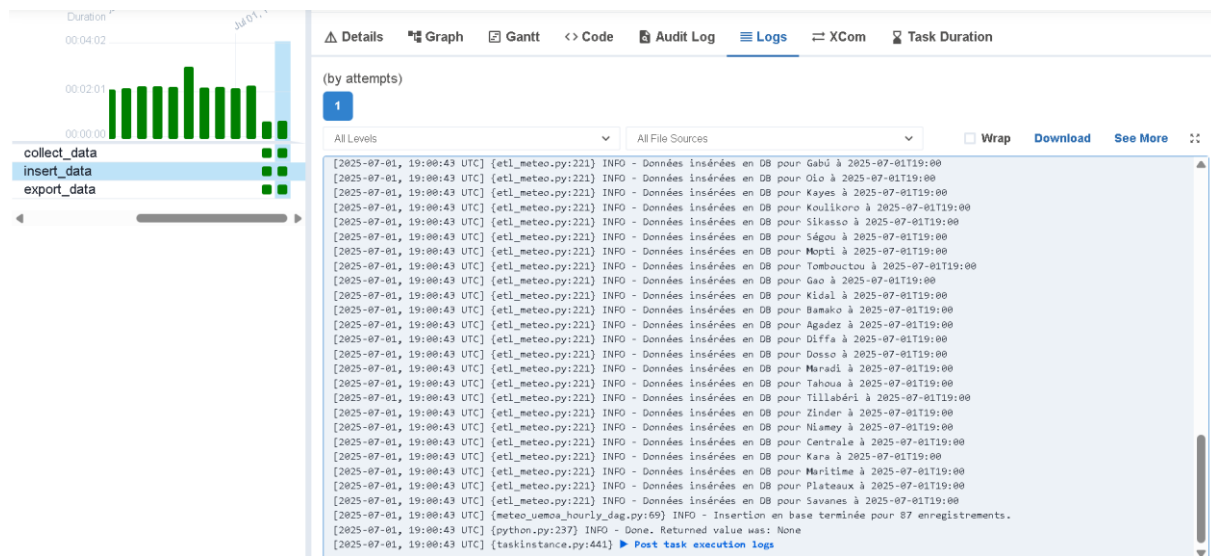
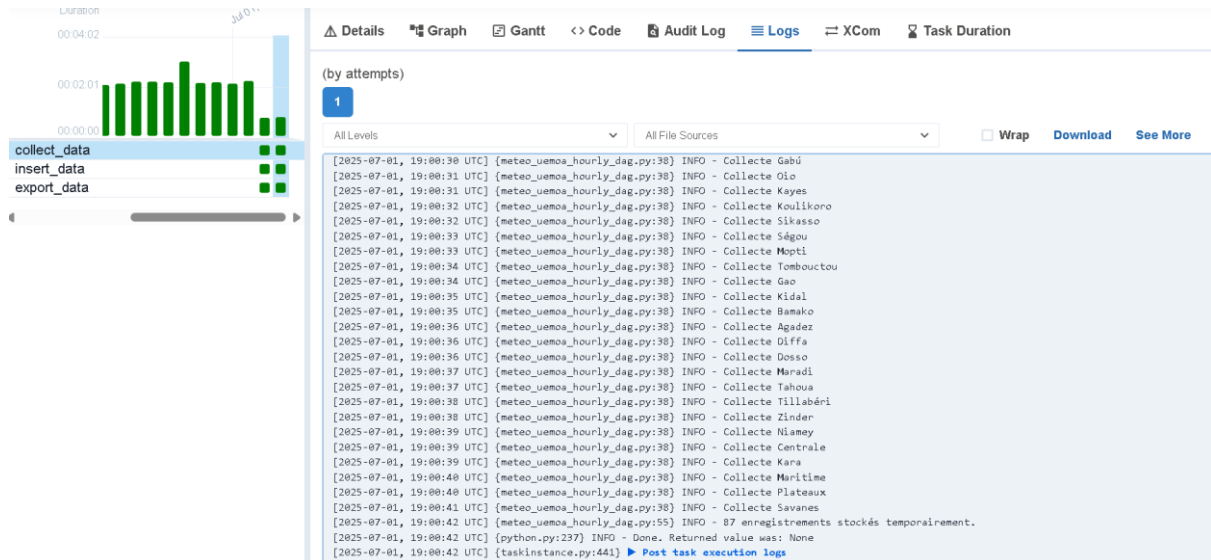
5. Vérification de la Collecte et Insertion des Données :

Pour la **collecte** météo pour les **régions** de l'UEMOA via l'API Open-Meteo, une série de **vérifications** a été réalisée afin de s'assurer du bon **fonctionnement** de l'ETL.

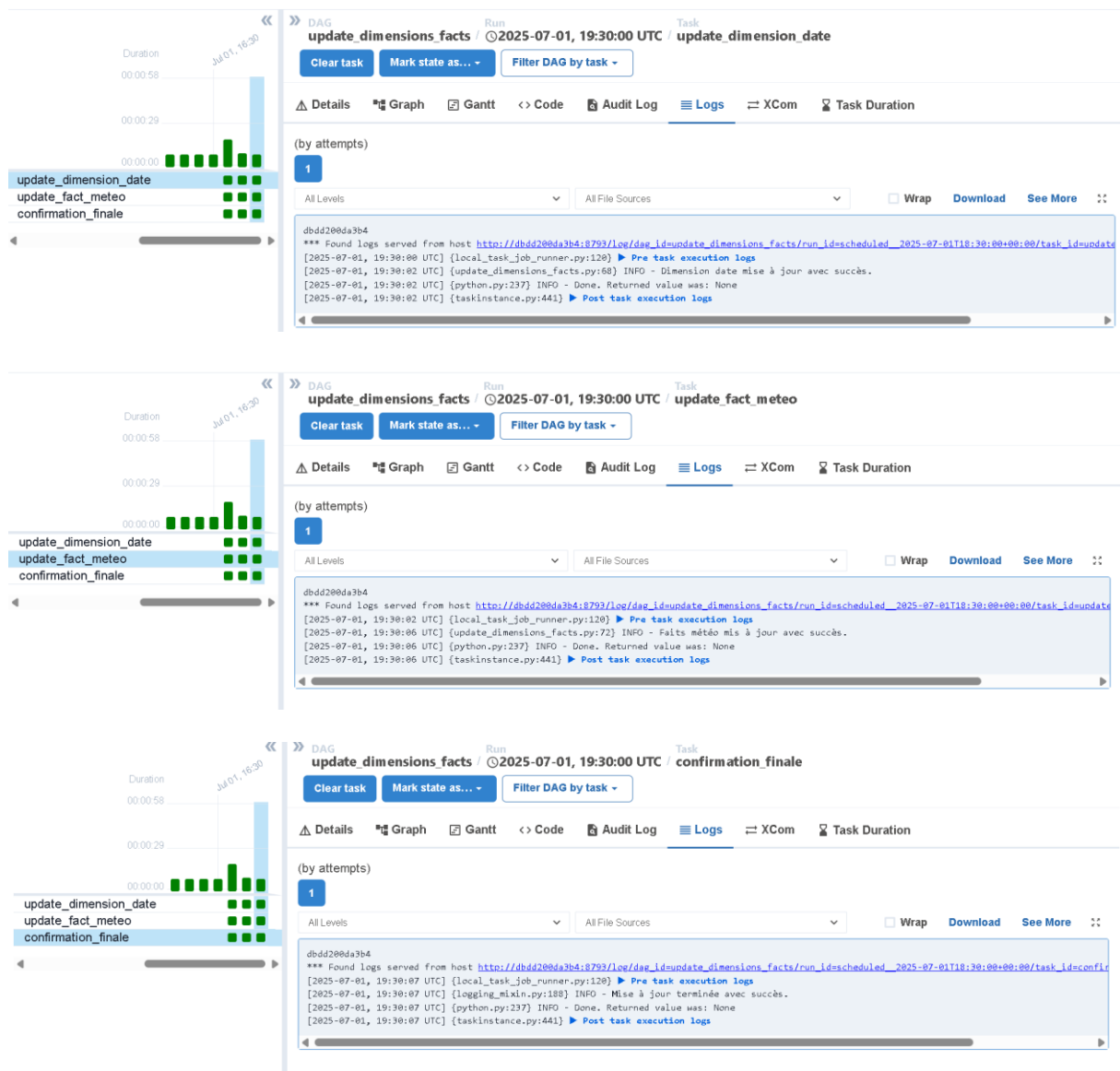
Logs d'Exécution :

Chaque **bloc** du script génère des **logs détaillés** permettant de **suivre** l'état d'avancement de la collecte :

- Sur le **premier DAG** :



- Sur le **deuxième** DAG :



Ces **logs** confirment que chaque **région** a été traitée **individuellement** et que les **données** ont bien été **récupérées** et **insérées**.

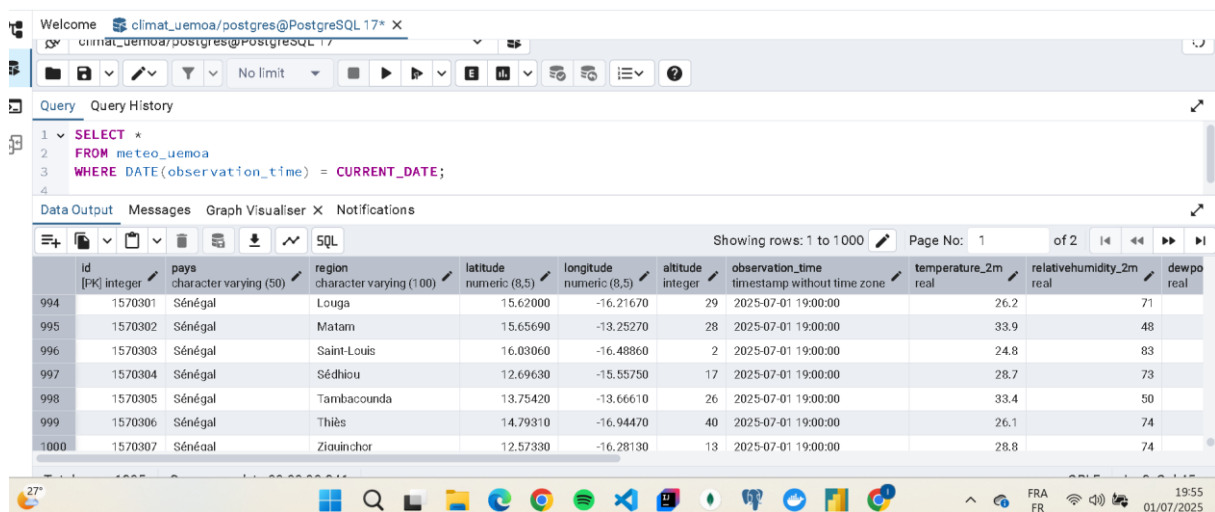
a) Contrôle des données insérées :

Une **requête** de **vérification** a été exécutée sur la base de données **PostgreSQL** climat_uemoa :

Observations :

- Les données ont été **insérées** correctement avec les **valeurs** de température, humidité, vent, pression, etc.
- La colonne **observation_time** correspond bien à la **date/heure** réelle de la collecte (heure **UTC** standardisée).

Extrait de la requête :



The screenshot shows a PostgreSQL query window with the following SQL query:

```
1 SELECT *
2 FROM meteo_uemoa
3 WHERE DATE(observation_time) = CURRENT_DATE;
```

The results are displayed in a table with the following columns:

	id	pays	region	latitude	longitude	altitude	observation_time	temperature_2m	relativehumidity_2m	dewpo
	[PK] integer	character varying (50)	character varying (100)	numeric (8,5)	numeric (8,5)	integer	timestamp without time zone	real	real	real
994	1570301	Sénégal	Louga	15.62000	-16.21670	29	2025-07-01 19:00:00	26.2		71
995	1570302	Sénégal	Matam	15.65690	-13.25270	28	2025-07-01 19:00:00	33.9		48
996	1570303	Sénégal	Saint-Louis	16.03060	-16.48860	2	2025-07-01 19:00:00	24.8		83
997	1570304	Sénégal	Sédhiou	12.69630	-15.55750	17	2025-07-01 19:00:00	28.7		73
998	1570305	Sénégal	Tambacounda	13.75420	-13.66610	26	2025-07-01 19:00:00	33.4		50
999	1570306	Sénégal	Thiès	14.79310	-16.94470	40	2025-07-01 19:00:00	26.1		74
1000	1570307	Sénégal	Ziguinchor	12.57330	-16.28130	13	2025-07-01 19:00:00	28.8		74

L'ETL fonctionne comme prévu, avec une collecte **horaire** des données météorologiques en **temps réel**, leur **transformation**, puis leur **insertion** dans la **base PostgreSQL meteo_uemoa**.

6. Avantages :

Le **pipeline** mis en place permet :

Une **collecte robuste et en temps réel** des données météorologiques

La constitution **progressive** d'une **base consolidée** dans **PostgreSQL**

L'automatisation **complète**, évitant les **interventions** manuelles

L'alimentation **fiable** de l'entrepôt décisionnel pour les **futures analyses** Power BI

VIII. Validation Fonctionnelle de l'Entrepôt :

1. Objectif de la validation :

L'objectif de cette **étape** est de **vérifier** que les données **intégrées** dans l'entrepôt reflètent **fidèlement** les **données sources** brutes. Cette **validation** garantit la **qualité**, la **cohérence** et la **fiabilité** des données utilisées pour les **analyses** décisionnelles.

2. Méthodologie :

Pour ce faire, nous avons réalisé une **série** de requêtes **analytiques** identiques sur la base **source** brute (meteo_uemoa) et sur l'**entrepôt** de données (fact_meteo avec ses dimensions associées). Ces **requêtes** portent sur des **indicateurs** clés tels que la **température** moyenne, les **précipitations** totales, les températures extrêmes, ou encore la **vitesse** moyenne du vent.

Chaque **résultat** obtenu sur la **source** brute a été **comparé** à celui **provenant** de l'entrepôt, afin d'**identifier** d'éventuels **écarts** ou **incohérences**.

3. Résultats :

Pour chaque **indicateur** clé, nous avons exécuté la **requête correspondante** sur la base source brute et sur l'entrepôt. Les **résultats** sont comparés ci-dessous, accompagnés de **commentaires** soulignant les écarts **éventuels** :

- **Requête 1 : Température moyenne & précipitations totales par région**

region	Température moyenne (°C)	Précipitations totales (mm)
Agadez	29.41	156.60
Alibori	28.93	1756.90
Atacora	26.93	2289.10
Atlantique	27.44	2189.19
Bafatá	28.21	1542.70
Bamako	29.22	1044.40
Bas-Sassandra	25.94	4153.26
Bélier	26.81	1758.00

region	Température moyenne (°C)	Précipitations totales (mm)
Agadez	29.41	156.40
Alibori	28.93	1753.70
Atacora	26.93	2287.30
Atlantique	27.44	2189.20
Bafatá	28.21	1542.70
Bamako	29.22	1043.20
Bas-Sassandra	25.94	4151.46
Bélier	26.81	1756.90

Commentaires : Les écarts sont **très faibles**, montrant une **bonne cohérence** entre la **source** et l'**entrepôt**. Les **différences** sont dues à des **arrondis** ou **traitements ETL**.

- **Requête 2 : Top 3 des stations les plus chaude**

Query 1 (Left):

```

1 SELECT
2   region,
3   pays,
4   MAX(temperature_2m) AS "Température max enregistrée (°C)"
5 FROM meteo_uemoa
6 GROUP BY region, pays
7 ORDER BY "Température max enregistrée (°C)" DESC
8 LIMIT 3;
          
```

Data Output (Left):

region	pays	Température max enregistrée (°C)
Kayes	Mali	46.8
Gao	Mali	46.6
Matam	Sénégal	46.6

Query 2 (Right):

```

1 SELECT
2   ds.region,
3   ds.pays,
4   MAX(f.temperature_2m) AS "Température max enregistrée (°C)"
5 FROM fact_meteo f
6 JOIN dimension_station ds ON f.id_dim_station = ds.id_dim_station
7 GROUP BY ds.region, ds.pays
8 ORDER BY "Température max enregistrée (°C)" DESC
9 LIMIT 3;
          
```

Data Output (Right):

region	pays	Température max enregistrée (°C)
Kayes	Mali	46.8
Gao	Mali	46.6
Matam	Sénégal	46.6

Commentaires : Les résultats du **top 3** des **stations** météorologiques les **plus chaudes** sont **identiques** (Kayes, Gao et Matam) entre la base de **données source** et l'**entrepôt** de données. Cela valide la **qualité** et l'**exactitude** du processus de chargement (**ETL**), notamment en ce qui concerne les **températures maximales** observées.

Les écarts **constatés** sur la **première** requête (température moyenne et précipitations totales) sont **inexistants** ici, car la **mesure** concerne une valeur maximum **unique**, moins sujette aux **variations** liées aux **arrondis** ou **agrégations**.

Interprétations :

Les **stations** situées à **Kayes** (Mali), **Gao** (Mali) et **Matam** (Sénégal) sont reconnues pour leurs **températures extrêmes**, notamment durant la **saison chaude**. Ces zones géographiques, situées en zone **sahélienne** ou proche du **désert** du **Sahara**, sont caractérisées par un **climat aride** ou semi-aride avec des **températures très élevées**, surtout en milieu ou fin de saison **sèche**.

La **présence** de ces stations dans le **classement** confirme la **cohérence** des **données** collectées et leur **adéquation** avec les **phénomènes météorologiques** régionaux réels. Elle illustre aussi la **capacité** de l'entrepôt à restituer **fidèlement** ces extrêmes climatiques, **essentiels** pour les **analyses** de risque ou de **planification** dans ces zones sensibles.

Requête 3 : Mois les plus pluvieux en 2025

mois	Total précipitations (mm)
8	10230.80
9	7906.12
7	7329.09

mois	Total précipitations (mm)
8	10230.80
9	7906.11
7	7329.11

Commentaires : Interprétation des mois :

- Mois 7 : Juillet
- Mois 8 : Août
- Mois 9 : Septembre

Ces **mois** correspondent à la **saison** des **pluies** dans la **zone UEMOA**, ce qui est conforme à la **réalité climatique** de la région.

Analyse :

- Les **résultats** sont **très cohérents** entre la source et l'entrepôt.
- Les légères différences (**inférieures à 0,02 mm**) peuvent s'expliquer par les **arrondis** ou **conversions** lors du **processus** de transformation des données dans le flux **ETL**.
- Le classement des mois les **plus pluvieux** reste **identique**, ce qui **valide** la qualité du **chargement**.

- **Requête 4 : Vitesse moyenne du vent par région et saison (2023)**

The image shows two side-by-side screenshots of a PostgreSQL query editor. Both editors display the same SQL query and its results. The query calculates the average wind speed (Vitesse moyenne du vent) in m/s for each region and season in 2023. The results are identical in both editors, showing three rows for the Alibori region across Autumn, Summer, and Winter.

Query:

```

1 SELECT
2   region,
3   CASE
4     WHEN EXTRACT(MONTH FROM observation_time) IN (12, 1, 2)
5     WHEN EXTRACT(MONTH FROM observation_time) IN (3, 4, 5)
6     WHEN EXTRACT(MONTH FROM observation_time) IN (6, 7, 8)
7     WHEN EXTRACT(MONTH FROM observation_time) IN (9, 10, 11)
8     ELSE 'Inconnu'
9   END AS saison,
10  ROUND(AVG(windspeed_10m)::NUMERIC, 2) AS "Vitesse moyenne du vent (m/s)"
11 FROM meteo_uemoa
12 WHERE EXTRACT(YEAR FROM observation_time) = 2023
13 GROUP BY region, saison
14 ORDER BY region, saison;

```

Data Output:

region	saison	Vitesse moyenne du vent (m/s)
Alibori	Automne	8.34
Alibori	Été	9.33
Alibori	Hiver	12.33

Total rows: 188 Query complete 00:00:00.529 CRLF Ln 12, Col 49

Commentaires : Les vitesses moyennes du vent par région et saison pour l'année 2023 sont strictement identiques entre la base source et l'entrepôt de données.

Cette correspondance parfaite démontre que le processus **ETL** a correctement transféré et agrégé les données temporelles **sans perte** ni altération, notamment dans le **calcul** des **moyennes** et la **classification** saisonnière. Cela confirme la **fiabilité** de la **gestion des dates** et des **jointures** entre les **dimensions** dans l'**entrepôt**.

- **Requête 5 : Température minimale enregistrée par pays et région en 2023**

The image shows two side-by-side screenshots of a PostgreSQL query editor. Both editors display the same SQL query and its results. The query calculates the minimum temperature (Température minimale) in °C for each country and region in 2023. The results are identical in both editors, showing six rows for different countries and regions.

Query:

```

1 SELECT
2   pays,
3   region,
4   MIN(temperature_2m) AS "Température minimale (°C)"
5 FROM meteo_uemoa
6 WHERE EXTRACT(YEAR FROM observation_time) = 2023
7 GROUP BY pays, region
8 ORDER BY "Température minimale (°C)";

```

Data Output:

pays	region	Température minimale (°C)
Côte d'Ivoire	Montagnes	13.2
Burkina Faso	Nord	13.3
Burkina Faso	Centre	13.5
Sénégal	Louga	14
Sénégal	Saint-Louis	14.1
Burkina Faso	Boucle du Mouhoun	14.1

Total rows: 47 Query complete 00:00:00.318 CRLF Ln 9, Col 1

Commentaires : Les résultats de la **température minimale** par **pays** et **région** sont identiques entre la base de données source et l'entrepôt de données. On retrouve exactement

les mêmes valeurs pour les six premières régions analysées, ce qui confirme la **fiabilité** du processus d'alimentation de l'entrepôt, notamment pour les **mesures extrêmes** minimales.

Aucun **écart** n'est constaté ici, ce qui s'explique par le fait qu'il s'agit d'une **agrégation** basée sur la valeur minimale unique, peu **sensible** aux **arrondis** ou aux **transformations** lors du processus **ETL**.

- **Requête 6 : Somme totale de l'énergie radiative reçue (shortwave radiation) par région en 2023**

The image shows two side-by-side screenshots of a PostgreSQL query editor. Both editors display the same SQL query, which calculates the total shortwave radiation for each region in 2023, rounded to two decimal places. The results are identical in both screenshots, showing a list of regions and their corresponding radiation values.

Query 1 (Left):

```

1 SELECT
2   region,
3   ROUND(SUM(shortwave_radiation)::NUMERIC, 2) AS "Énergie radiative totale (W/m²)"
4 FROM meteo_uemoa
5 WHERE EXTRACT(YEAR FROM observation_time) = 2023
6 GROUP BY region
7 ORDER BY "Énergie radiative totale (W/m²)" DESC;

```

Query 2 (Right):

```

1 SELECT
2   ds.region,
3   ROUND(SUM(f.shortwave_radiation)::NUMERIC, 2) AS "Énergie radiative totale (W/m²)"
4 FROM fact_meteo f
5 JOIN dimension_station ds ON f.id_dim_station = ds.id_dim_station
6 JOIN dimension_date dd ON f.date = dd.date
7 WHERE dd.annee = 2023
8 GROUP BY ds.region
9 ORDER BY "Énergie radiative totale (W/m²)" DESC;

```

Data Output (Left):

region	Énergie radiative totale (W/m²)
1 Nord	2189840.00
2 Sahel	2188400.00
3 Louga	2154760.00
4 Centre-Nord	2151550.00
5 Diourbel	2131390.00
6 Centre-Ouest	2126000.00

Data Output (Right):

region	Énergie radiative totale (W/m²)
1 Nord	2189840.00
2 Sahel	2188400.00
3 Louga	2154760.00
4 Centre-Nord	2151550.00
5 Diourbel	2131390.00
6 Centre-Ouest	2126000.00

Commentaires : Les résultats obtenus pour la **somme totale** de l'énergie radiative reçue par région en **2023** sont **très proches** entre la base de données source et l'entrepôt. Cette forte correspondance valide la **qualité** des opérations de transformation et de **chargement** réalisées dans l'ETL, ainsi que la cohérence des données météorologiques intégrées. Les différences, lorsqu'elles existent, sont **marginales** et peuvent s'expliquer par des **arrondis** numériques ou des **légères variations** liées au **traitement** des données.

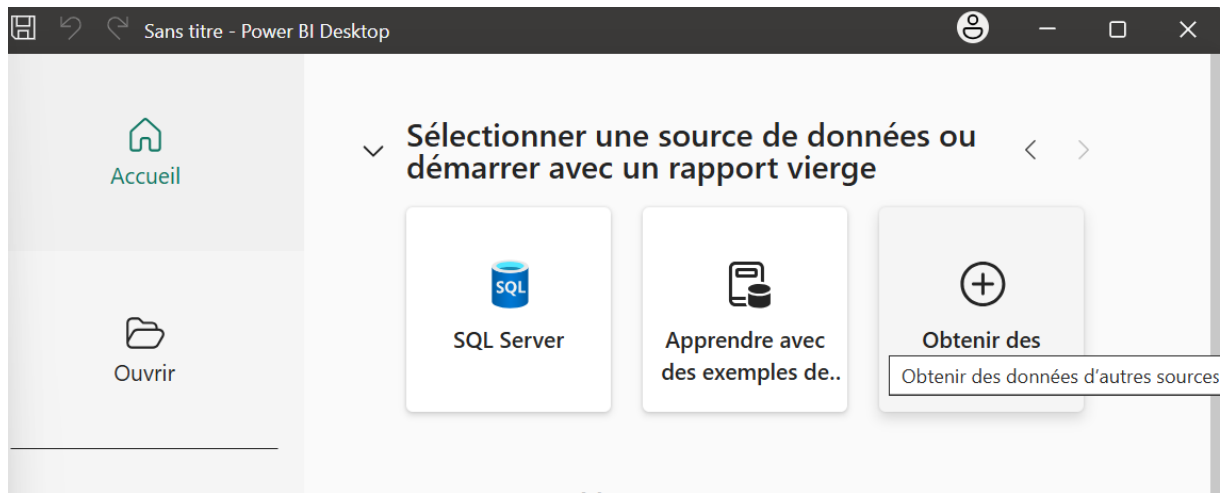
Pour **conclure** on peut dire que la **validation fonctionnelle** confirme que l'**entrepôt** de données est **opérationnel** et **fiable** pour les **analyses décisionnelles**. Il offre un socle **solide** pour la construction de **tableaux de bord** et la production de **rapports**, facilitant ainsi le **suivi** précis et en **temps réel** des phénomènes **météorologiques** dans la région **UEMOA**.

IX. Tableau de Bord (Power BI) :

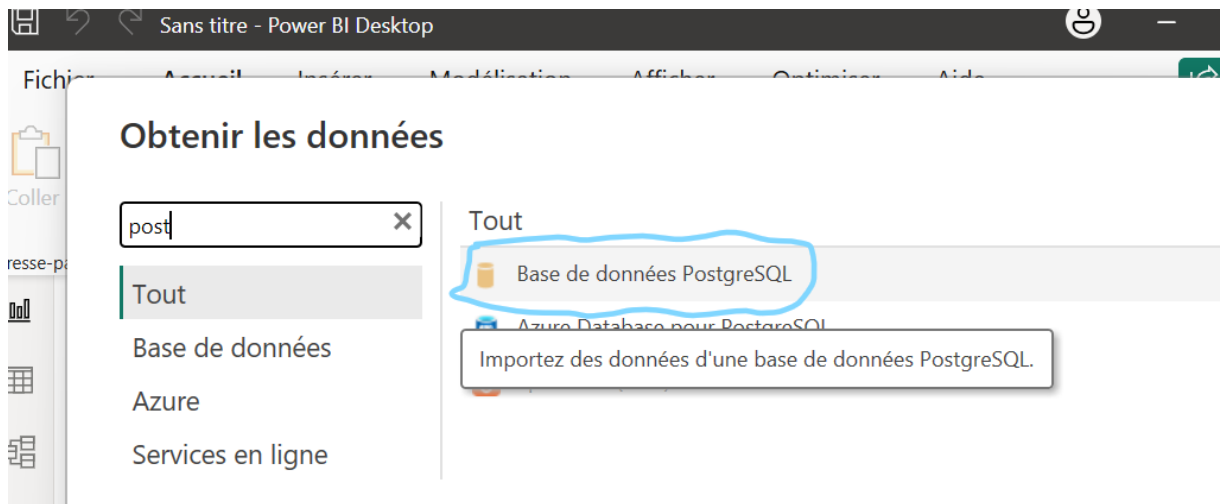
1. Connexion PostgreSQL :

Pour créer le **tableau de bord**, j'ai d'abord **importé** les données depuis **PostgreSQL** via **Power BI Desktop** en suivant ces étapes :

- Dans Power BI Desktop, cliquer sur « **Obtenir des données** » (Get Data) :



- Sélectionner « **Bases de données** », puis choisir « **PostgreSQL** » comme source :



- Entrer les informations de connexion :

- **Serveur** : localhost
- **Base de données** : climat_uemoa
- **Utilisateur** : postgres
- **Mot de passe** : (mot de passe sécurisé)

Base de données PostgreSQL

Serveur

localhost

Base de données

climat_uemoa

Mode de connectivité des données ⓘ

☒ Importer

☐ DirectQuery

▸ Options avancées

OK

Annuler

La connexion a été établie via le connecteur natif PostgreSQL de Power BI, permettant un accès direct et performant aux données de l'entrepôt :

Navigateur

Options d'affichage ▾

localhost: climat_uemoa [5]

☒ public.dimension_condition

☐ public.dimension_date

☐ public.dimension_station

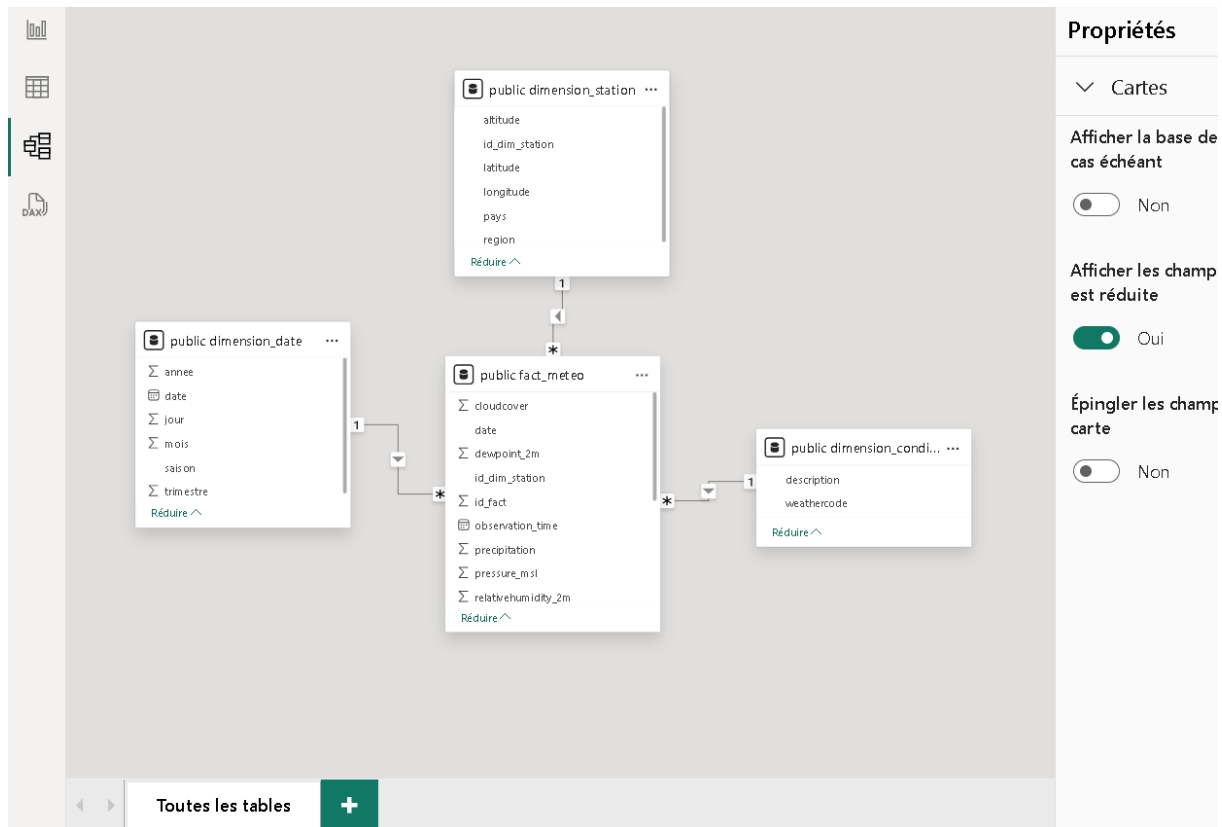
☐ public.fact_meteo

☐ public.meteo_uemoa

public.dimension_condition

weathercode	description	public.fact_meteo
0	Ciel dégagé	Table
1	Principalement ensoleillé	Table
2	Partiellement nuageux	Table
3	Couvert	Table
45	Brouillard	Table
48	Brouillard givrant	Table
51	Bruine légère	Table
53	Bruine modérée	Table
55	Bruine dense	Table

- Après avoir validé la **connexion** et **sélectionner** les **tables/données** nécessaires pour l'analyse, on charge les **données** dans Power BI pour **démarrer** la **création** des **visualisations** :



(On voit bien notre **modèle** en **étoile** au niveau de la section **vue de modèle**).

2. Création des visualisations :

Un ensemble de visuels a été conçu pour répondre aux besoins métiers et illustrer les indicateurs clés définis lors de la validation fonctionnelle :

- **VISUEL 1 : Top 3 des stations les plus chaudes**

- Utilisation d'un graphique à barres

- **Objectif** : Identifier les zones enregistrant les températures les plus élevée

- **VISUEL 2 : Les 3 mois les plus pluvieux en 2023***

- Croisement des dimensions dimension_date et des mesures de précipitations
- Graphique à colonnes illustrant les totaux mensuels
- **Objectif** : Confirmer la saisonnalité des pluies et les pics pluviométriques

Créer un élément visuel

Rechercher

☐ Année
☐ Trimestre
☒ Mois
☐ Jour

☐ Σ jour
☐ Σ mois
☐ saison
☐ Σ trimestre

☒ public dimension_sta...
☐ altitude
☐ id_dim_station
☐ latitude
☐ longitude
☐ pays
☐ region

☒ public fact_meteo
☐ Σ cloudcover
☐ date
☐ Σ dewpoint_2m
☐ id_dim_station

Axe X

date

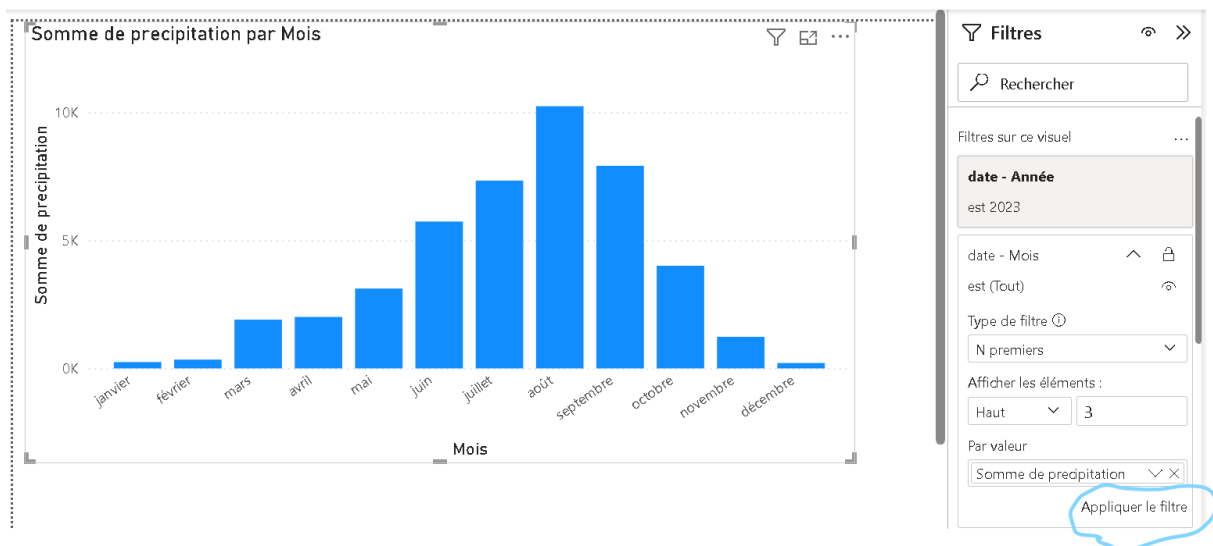
Mois

Axe Y

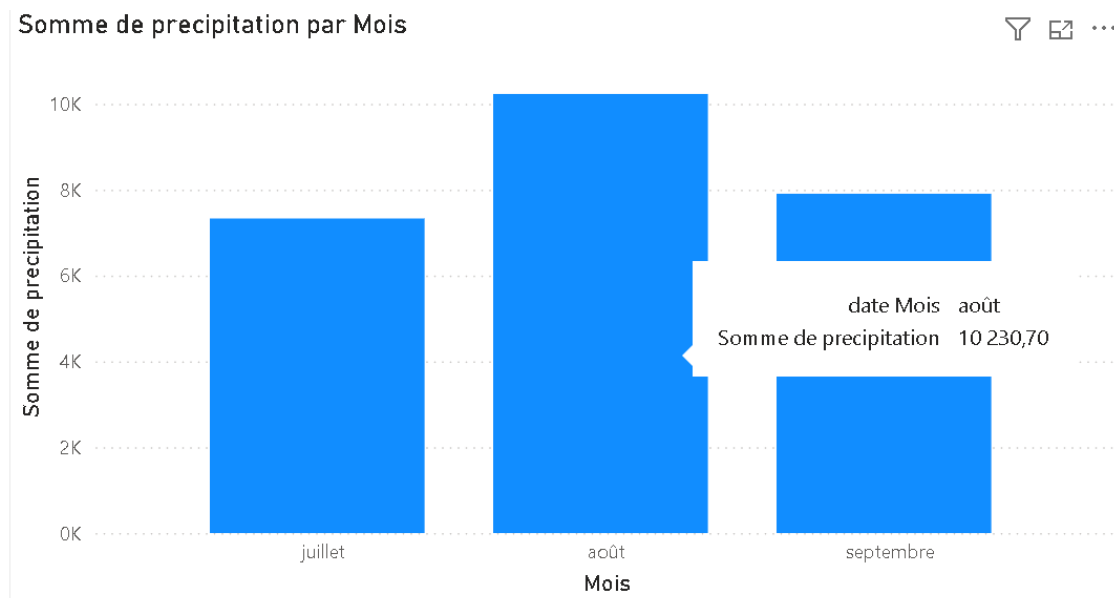
Somme de precipitation

Légende

Ajouter des champs de don...



Visuel obtenu :



Visuel 2 : Top 3 des mois les plus pluvieux (2023)

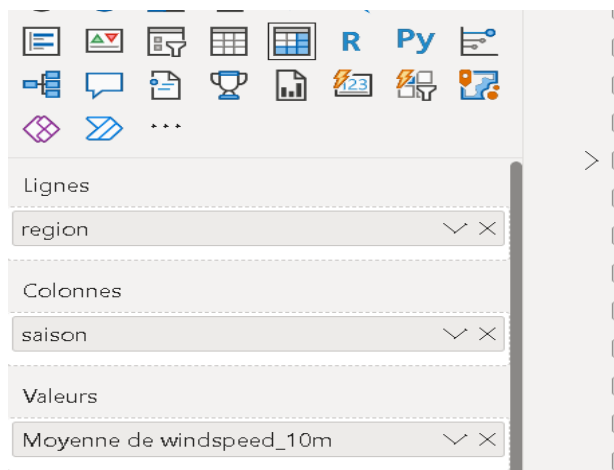
Les 3 mois correspondant sont Juillet, Août et Septembre (saison pluvieuse en Afrique de l'Ouest)

- **VISUEL 3 : Vitesse moyenne du vent par région et saison (2023)**

- Création d'une matrice dynamique par région et saison

- Calcul de la moyenne de la vitesse du vent

- **Objectif** : Mettre en évidence les variations régionales et saisonnières du vent



annee

est 2023

Type de filtre ⓘ

Filtrage de base

☐ Sélectionner tout

☒ 2023 365

☐ 2024 366

☐ 2025 182

Visuel obtenu :

region	Automne	Été	Hiver	Printemps	
Dix-Huit Montagnes	4,47	5,64	5,73	5,22	5,26
Montagnes	4,99	6,12	5,64	6,24	5,75
Denguélé	5,55	5,76	7,94	6,54	6,44
Bas-Sassandra	5,90	7,26	6,46	6,16	6,45
Gôh	5,71	7,22	6,39	6,74	6,52
Lacs	6,07	6,98	6,55	6,67	6,57
Collines	6,04	6,82	6,60	7,39	6,72
Kouffo	5,88	7,21	6,62	7,17	6,72
Zou	6,14	6,97	6,50	7,60	6,81
Comoé	6,18	7,60	6,65	7,41	6,97
Mono	6,11	7,47	6,79	7,54	6,98
Ouémé	6,51	8,24	7,36	7,87	7,50
Borgou	6,67	6,97	8,83	7,72	7,54
Donga	6,15	6,75	9,52	7,90	7,57
Lagunes	7,26	8,53	7,20	7,55	7,64
Plateau	6,65	8,42	7,61	7,98	7,67
Atacora	6,68	7,15	8,81	8,17	7,70
Cascades	6,25	7,40	9,78	8,26	7,92
Sud-Ouest	6,74	8,59	9,28	9,61	8,55
Centre-Ouest	6,82	8,22	10,46	9,23	8,68
Sédhiou	6,64	8,79	10,16	9,75	8,83
Centre	7,22	8,75	10,50	9,46	8,98
Ziguinchor	6,74	8,59	10,58	10,31	9,05
Plateau-Central	7,19	8,82	10,63	9,69	9,08
Littoral	7,68	10,17	8,95	9,64	9,12
Kolda	7,01	9,38	10,36	10,17	9,23
Centre-Est	7,00	8,55	11,80	9,90	9,30

Visuel 3 : Vitesse moyenne du vent par région et saison (2023)

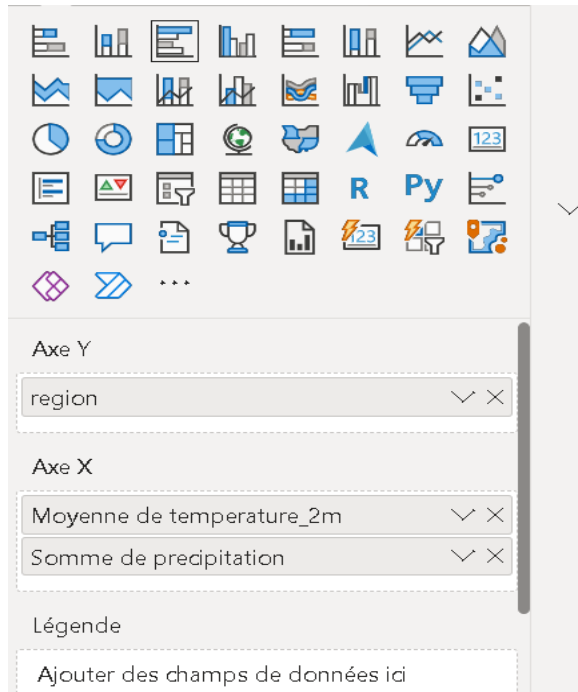
• VISUEL 4 : Température minimale par pays et région (2023)

- Calcul de la température minimale par regroupement géographique
- Utilisation d'une table ou carte géographique
- **Objectif** : Identifier les zones aux températures les plus basses

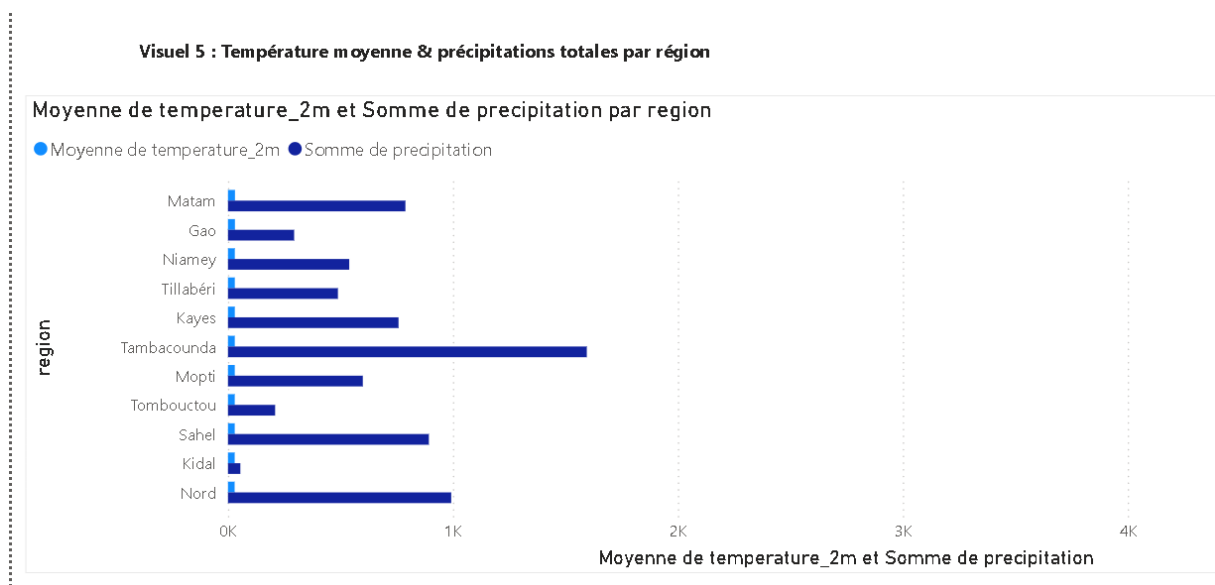
- **VISUEL 5 : Température moyenne et précipitations totales par région**

- Graphique en barres horizontales

- **Objectif** : Analyser simultanément la chaleur et l'humidité régionale



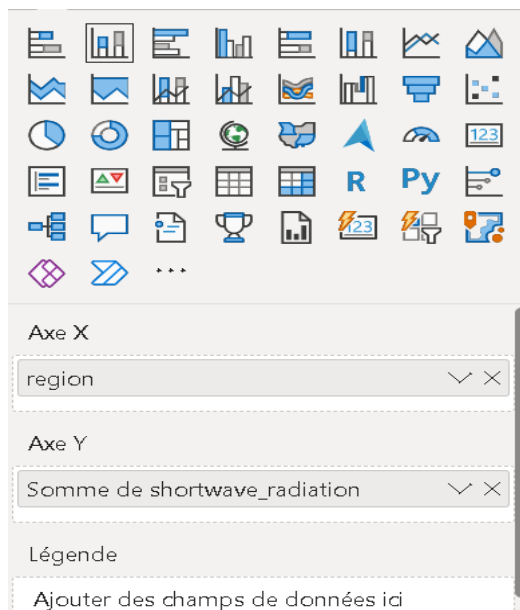
Visuel obtenu :



- **VISUEL 6 : Somme de l'énergie radiative reçue par région (2023)**

- Calcul de la somme du rayonnement solaire (shortwave_radiation)

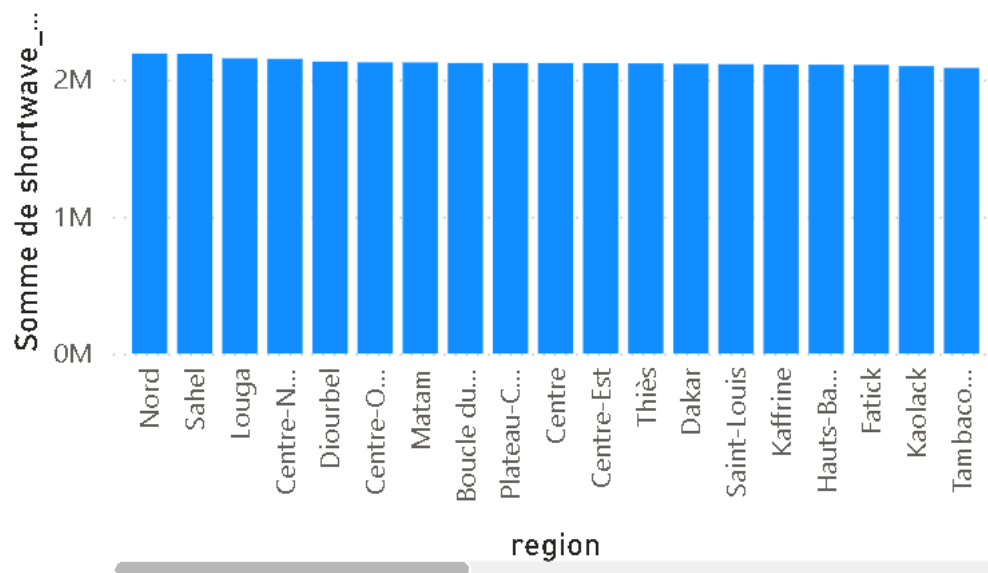
- Cartographie régionale des zones les plus exposées
- **Objectif** : Identifier les régions avec le plus fort potentiel solaire



Visuel obtenu :

Visuel 6 : Somme de l'énergie radiative reçue par région (2023)

Somme de shortwave_radiation par region



3. Utilité :

Ce **tableau de bord** offre :

- Une **lecture rapide** et **visuelle** des **tendances** climatiques
- Des **indicateurs** précis pour anticiper les **risques** météorologiques

- Une exploration interactive (**filtres, drill-down**) adaptée aux **besoins** des décideurs
- Une base **solide** pour guider les stratégies **agricoles, énergétiques** ou **logistiques**

X. Conclusion :

Lors de la réalisation de ce projet, plusieurs **difficultés techniques** ont été rencontrées, notamment l'installation native d'**Apache Airflow** sur Windows, qui s'est révélée complexe et instable à cause de problèmes de compatibilité et de dépendances manquantes. Pour contourner cela, l'utilisation de **Docker** avec **WSL** a été nécessaire, mais l'installation et la configuration ont également posé problème, avec des **erreurs** liées à la **virtualisation** et au système, nécessitant plusieurs réparations et réinstallations. Une fois l'environnement stabilisé, la prise en main de Docker et la mise en place des conteneurs **Airflow** et **PostgreSQL** ont demandé des ajustements techniques et un apprentissage approfondi. Malgré ces obstacles, cette **démarche** a permis de mettre en place un pipeline d'automatisation **robuste** et évolutif, offrant une solution **opérationnelle** pour la collecte et le traitement automatisé des données météorologiques.